17 DEZEMBRO 2025

FORMALIZAÇÃO DE MÁQUINAS DE ESTADOS FINITOS COMPONÍVEIS EM HASKELL: SEGURANÇA DE TIPOS E COMPLETUDE FUNCIONAL VIA MONADS E TIPOS ALGÉBRICOS

PETERSON CARARA

Conceitos abordados

Máquinas de Estados Finitos (FSM)

É o modelo matemático fundamental projeto. Uma FSM consiste num conjunto finito de Estados, um conjunto de Eventos (entradas) e uma Função de Transição que define como o sistema muda de um estado para outro quando um evento ocorre.

- No projeto: Não está apenas implementando uma FSM, mas formalizando-a para que ela seja componível e segura.

Conceitos abordados

Tipos Algébricos de Dados (ADTs)

Em Haskell, os ADTs (especificamente os Tipos Soma) permitem definir estados e eventos como tipos de dados explícitos.

- Por que é importante: Em vez de usar Strings ou Integers (que podem assumir valores inválidos), você define `data Estado = Aberto | Fechado`.
- O que isso permite: Torna-se impossível para o sistema entrar num estado que não foi definido. O compilador garante que apenas os símbolos do seu modelo existam.

Conceitos abordados

Segurança de Tipos (Type Safety) e "Invalidez de Tipagem

Este é o conceito de usar o sistema de tipos para impor regras de negócio.

- Segurança: Se uma transição de "Fechado" para "Processando" é proibida pela lógica do negócio, você codifica essa restrição no sistema de tipos (usando técnicas avançadas como as vistas no framework CREM)
- Teste de Invalidez: É a prova real do projeto. Consiste em tentar escrever um código que viole a regra da máquina. Se o código não compilar, a segurança de tipos foi atingida.

Conceitos abordados

Completude Funcional via Pattern Matching

O Haskell exige que, ao tratar um ADT, você lide com todos os seus construtores (casos).

- Aplicação: Na função de transição da FSM, o Pattern Matching Exaustivo força o programador a definir o que acontece para cada combinação de (Estado × Evento).
- Resultado: Elimina os "bugs de silêncio" onde o sistema ignora um evento por esquecimento do desenvolvedor.

Conceitos abordados

Monads (especificamente a State Monad)

Uma Monad é uma estrutura que permite sequenciar computações com "efeitos".

- State Monad: Modela uma função `s → (a, s)`, onde `s` é o estado. Ela permite que a FSM "evolua" ao longo do tempo de forma pura, sem recorrer a variáveis mutáveis globais.
- Pureza: Isso garante que a única forma de alterar o estado é através da função de transição formalmente definida.

Conceitos abordados

Teoria das Categorias e Categorias de Kleisli

A Teoria das Categorias estuda estruturas matemáticas e as relações (setas) entre elas.

- Componibilidade: O projeto foca em como unir FSMs pequenas para formar sistemas maiores.
- Categoria de Kleisli: É o formalismo que permite compor funções monádicas. No meu caso, ela fornece a "cola" matemática para garantir que, ao juntar duas máquinas de estados, o resultado seja logicamente consistente e previsível.

Conceitos abordados

Funtores (Functors): A Base da Composição

Em Haskell, um Functor é uma classe de tipos que permite aplicar uma função a um valor dentro de um contexto, sem alterar o contexto em si.

- Definição Matemática: É um mapeamento entre categorias que preserva a estrutura das setas e a composição.
- Aplicação na FSM: Permite transformar o dado de saída de uma máquina de estados (como um log ou um ID) sem interferir na lógica de transição de estado.

FUNTORES

A Função fmap (ou < $ >)

A operação principal de um Functor é o fmap. Sua assinatura é:

```
fmap::(a → b) → f a → f b
```

- a → b: Uma função pura (ex: converter um ID de estado para String).
- f a: O valor dentro do contexto (ex: um Maybe Estado ou State s a).
- f b: O resultado no mesmo contexto.

FUNTORES

Por que Funtores são Úteis na FSM?

- Abstração de Erros: Se a sua FSM usa Maybe ou Either para lidar com falhas, o Functor permite processar o sucesso sem escrever múltiplos blocos if/else ou case manuais.
- Imutabilidade: O Functor garante que, ao transformar o valor, você não está alterando o estado original da máquina "na força bruta", respeitando a pureza funcional.
- Composição Horizontal: Facilita a criação de "peças menores" que podem ser combinadas posteriormente via Categorias de Kleisli.

Conceitos abordados

Domain-Driven Design (DDD) e Agregados

Embora seja um conceito de arquitetura, o projeto (especialmente via framework CREM) utiliza o DDD para identificar as fronteiras da lógica de negócio.

- As FSMs são usadas para modelar o ciclo de vida dos Agregados do domínio, garantindo que as transições de estado do software reflitam exatamente as regras do mundo real.

O QUE É POSSIVEL(METODOS TRADICIONAIS):

Garantia de Corretude e Segurança de Tipos (Type Safety)

A principal vantagem é mover a verificação da lógica de transição da fase de testes (tempo de execução) para a fase de compilação (tempo de desenvolvimento)

RECURSO EM HASKELL:

Tipos Algébricos de Dados (ADTs)

Estados e Eventos Válidos Codificados em Tipos: O conjunto finito de estados e eventos da FSM é mapeado diretamente para um data type em Haskell (um Tipo Soma). Isso torna impossível que um estado ou evento não especificado exista no sistema.

RECURSO EM HASKELL:

Pattern Matching Exaustivo

Completude Funcional Imbatível: A função de transição (fsmTransition :: State → Event → State) deve usar pattern matching para cobrir todas as combinações possíveis de (Estado Atual × Evento). Se o programador omitir uma combinação, o compilador (GHC) emite um aviso ou erro, garantindo que não haja transições não tratadas.

RECURSO EM HASKELL:

Teste de Invalidez de Tipagem

Prevenção de Transições Proibidas: Ao formalizar as regras do negócio no tipo de dados (possivelmente usando recursos de Type-Level como no CREM), o compilador é capaz de recusar qualquer tentativa de codificar uma transição que seja inválida pela lógica da FSM (ex: tentar fazer Logout a partir de um estado LoggedOut).

O QUE É POSSIVEL(METODOS TRADICIONAIS):

Gestão Pura e Imutável de Estado via Monads

O estado em Haskell é sempre imutável. O conceito de "mudar de estado" é abstraído através do conceito de Mónade.

RECURSO EM TEÓRICO:

State Monad (Mónade de Estado)

Sequenciação de Cálculos de Estado Puros: A State Monad encapsula a função de transição de forma pura, permitindo que a lógica de "leitura, cálculo e escrita" do estado seja sequenciada. Isso permite que a FSM seja executada em uma sequência de eventos, mantendo o controle da mudança de estado sem variáveis mutáveis globais.

RECURSO EM TEÓRICO:

Imutabilidade Intrínseca

Rastreabilidade e Pura Matemática: Cada transição de estado é uma função pura que recebe o estado antigo e retorna o estado novo. Isso elimina efeitos colaterais e torna o comportamento da FSM completamente previsível e testável.

O QUE É POSSIVEL(METODOS TRADICIONAIS):

Composição Segura e Modularidade (via Teoria das Categorias)

O formalismo da Teoria das Categorias oferece um método elegante para construir FSMs complexas a partir de peças menores.

RECURSO TEÓRICO:

Composição de FSMs (Monadic Composition)

Arquiteturas Componíveis: O uso de Monads e, em particular, das Categorias de Kleisli, permite que a lógica de duas FSMs independentes seja combinada (composta) de forma segura. Isso é essencial para arquiteturas de software baseadas em Domain-Driven Design (DDD) onde múltiplos agregados de domínio (FSMs) precisam interagir.

RECURSO TEÓRICO:

Composição Horizontal

Reutilização e Modularidade: É possível combinar FSMs que operam em diferentes partes do estado do sistema de forma modular, sem que uma máquina precise saber os detalhes internos da outra (o que é crucial para o acoplamento fraco).

Vantagem da Abordagem Haskell	Fundamento (Por que é Possível)	Benefício Final
Verificação Estática Total	**ADTs + Pattern Matching Exaustivo:** O compilador valida toda a lógica de transição e garante que nenhum estado/evento foi esquecido.	Redução drástica de *bugs* em produção.
Gestão Pura de "Efeitos"	**State Monad:** O estado é encapsulado e manipulado por funções puras, em vez de variáveis mutáveis.	Código mais previsível, mais fácil de testar (unidade) e com alta rastreabilidade.
Composição Segura	**Teoria das Categorias (Kleisli):** O formalismo matemático define como FSMs independentes podem ser combinadas de forma correta.	Criação de arquiteturas complexas e escaláveis, mantendo a modularidade (DDD).
Domínio Codificado no Tipo	**Tipagem Avançada (Type-Level):** As regras de negócio (transições permitidas) são impostas na **assinatura de tipo**.	A regra do negócio é um **fato** no código, e não apenas uma convenção.

O QUE TEORICAMENETE NÃO É POSSIVEL(METODOS TRADICIONAIS):

Garantir a Corretude Lógica via Compilação (Static Logic Validation)

Em linguagens imperativas, o compilador verifica a sintaxe, mas é agnóstico à lógica do domínio.

- Limitação: Não é possível impedir que um programador codifique uma transição inválida (ex: ir do estado Fechado diretamente para Processando). Para o compilador tradicional, se ambos os estados forem do tipo Enum ou Int, a atribuição é válida.
- Consequência: A verificação da lógica de negócio é "empurrada" para os testes unitários ou, pior, para o tempo de execução (runtime), onde falhas podem causar comportamentos imprevisíveis.

Impedir "Estados Impossíveis" (Type Safety)

Representações comuns usam identificadores de baixo nível para estados e eventos.

- A Limitação: Se estados são representados por Strings ou Inteiros, teoricamente não é possível garantir que a variável de estado contenha apenas valores válidos. O sistema pode assumir um estado 99 ou "NULL", que não existe no modelo matemático da FSM.
- O Contraste com o Projeto: Sem o uso de Tipos Algébricos de Dados (ADTs), o sistema de tipos é fraco demais para "proibir" o erro por construção.

Assegurar a Completude Funcional (Exaustividade)

A transição de uma FSM é uma função que mapeia um par (Estado × Evento) para um novo Estado.

- A Limitação: Em linguagens imperativas (usando switch-case ou if-else), não é possível forçar o desenvolvedor a tratar todas as combinações possíveis. É comum esquecer de definir o que acontece quando um evento específico ocorre em um determinado estado.
- Consequência: O sistema pode simplesmente "ignorar" eventos ou cair em blocos default genéricos que escondem falhas de design, violando o princípio da completude funcional que seu projeto busca.

Isolar o Estado de Interferências Externas (Purity & Encapsulation)

Nas linguagens tradicionais, o estado da máquina é tipicamente uma variável mutável dentro de um objeto.

- A Limitação: Teoricamente, não é possível garantir que a única forma de alterar o estado seja através da função de transição definida. Qualquer método com acesso ao campo mutável pode alterá-lo indevidamente.
- Vantagem da minha Abordagem: Em Haskell, o estado é imutável. A "mudança" só ocorre via State Monad, garantindo que o fluxo seja estritamente controlado e matematicamente rastreável.

Composição de Máquinas com Prova Matemática

Sistemas complexos exigem que várias FSMs interajam.

- A Limitação: Em abordagens ad-hoc, a composição é feita por acoplamento manual. Não existe um formalismo que garanta que a união de duas máquinas resulte em uma terceira máquina logicamente consistente.
- O Que Falta: Falta a base da Teoria das Categorias (como as Categorias de Kleisli), que o projeto utiliza para permitir a composição segura. Sem isso, a modularidade é frágil e propensa a erros de sincronismo.

Resumo: O que não é possivel(Tradicional)

Validar lógica no compilador

Erradicar estados inexistentes

Obrigar tratamentos de cada caso

Garantir imutabilidade de estado

Compor FSMs de forma algébrica

Resumo: Riscos envolvidos

Transições proibidas chegam à produção

O sistema entra em "limbo" ou estados nulos.

Eventos são ignorados silenciosamente (Bugs).

Alterações externas "quebram" a lógica da FSM.

Integração de módulos é frágil e complexa.

O QUE TEORICAMENTE É POSSIVEL(FORMALIZAÇÃO EM HASKELL):

Garantir a "Corretude por Construção" (Type Safety)

Ao contrário das linguagens imperativas, onde estados são variáveis soltas, na sua formalização os estados e eventos são Tipos Algébricos de Dados (ADTs).

- O que é possível: Tornar fisicamente impossível que o sistema assuma um estado que não foi definido no modelo. Se o seu tipo define apenas Aberto e Fechado, não existe memória ou lógica no programa capaz de gerar um terceiro estado "fantasma" ou nulo.

Impor a Completude Funcional (Pattern Matching)

A formalização utiliza o Pattern Matching Exaustivo do Haskell para a função de transição.

- O que é possível: Forçar o compilador a verificar se você definiu o que acontece para todas as combinações de Estado x Evento.
- Impacto: Se adicionar um novo evento ao sistema e esquecer de atualizar uma parte da lógica, o código não compilará. Isso elimina os "bugs de esquecimento" comuns em abordagens ad-hoc.

Proibir Transições Inválidas em Tempo de Compilação

Este é o ponto central do "Teste de Invalidez de Tipagem".

- O que é possível: Codificar as regras de negócio diretamente nos tipos (usando GADTs ou o framework CREM). Se a regra diz que "não se pode ir de A para C", qualquer tentativa de escrever código que faça essa transição resultará num erro de compilação.
- A Prova: Você pode provar a segurança do seu sistema tentando "quebrá-lo" e mostrando que o compilador impede a criação do binário.

Isolar o Estado de Efeitos Colaterais (State Monad)

Utilizando a State Monad, é separado a lógica de transição pura da manipulação do estado.

- O que é possível: Garantir que o estado da máquina é imutável e que a única forma de "evoluir" o sistema é através de funções puras controladas. Isso elimina interferências externas acidentais (variáveis globais sendo alteradas por outras partes do código).

Composição Segura de Máquinas (Categorias de Kleisli)

Esta é a parte mais avançada da formalização, baseada na Teoria das Categorias.

- O que é possível: Unir duas ou mais FSMs independentes (ex: uma de Autenticação e uma de Carrinho) para criar uma máquina maior e mais complexa, garantindo que a união preserva todas as propriedades de segurança das máquinas originais.
- O formalismo: Através das Setas de Kleisli, a composição deixa de ser um "ajuste manual" (ad-hoc) e passa a ser uma operação algébrica previsível.

RESUMO DO PODER DESSA FORMALIZAÇÃO:

Funcionalidade	Como o Haskell viabiliza	Resultado Prático
Segurança Total	ADTs (Tipos Soma)	Zero estados "limbo" ou nulos.
Zero Omissões	Pattern Matching	Todos os cenários são obrigatoriamente tratados.
Regras Rígidas	Verificação Estática	Transições proibidas nem sequer chegam a ser executadas.
Modularidade	Kleisli Composition	Facilidade em escalar o sistema sem criar "código esparguete".
Rastreabilidade	Pureza Funcional	Comportamento 100% determinístico e fácil de testar.

Em suma: Nessa formalização, a máquina de estados não é apenas uma documentação que o código tenta seguir; o código é a própria especificação matemática, tornando o erro lógico uma impossibilidade técnica durante o desenvolvimento.

O QUE TEORICAMENTE NÃO É POSSIVEL(FORMALIZAÇÃO EM HASKELL):

Garantir o Término (O Problema da Parada)

Haskell é uma linguagem Turing-completa. Mesmo com uma FSM formalizada, o compilador não consegue garantir que uma sequência de transições não entrará num loop infinito.

- Limitação: Se a sua lógica de transição envolver recursividade não estruturada ou depender de condições externas complexas, a FSM pode "travar" numa computação infinita.
- Por que: O sistema de tipos do Haskell foca na corretude da transição, mas não na decidibilidade do término.

Prova Total de Valores de Runtime (Sem Tipos Dependentes)

Embora use ADTs para estados e eventos, o Haskell (GHC) padrão não possui Tipos Dependentes (como em linguagens como Agda ou Coq).

- Limitação: É difícil fazer com que o tipo de um dado dependa de um valor calculado em tempo de execução de forma trivial. Por exemplo, garantir via tipos que "esta transição só ocorre se o saldo for exatamente o valor X" é complexo em Haskell puro e exige extensões avançadas (como GADTs e Type Families).
- Consequência: Algumas validações de "valor de negócio" ainda precisam ocorrer dentro da função de transição e podem falhar em runtime se não forem tratadas com Maybe ou Either.

Controle Absoluto sobre Efeitos Externos (A Fronteira do I/O)

A formalização garante a pureza da lógica da FSM dentro da State Monad. No entanto, a FSM não existe no vácuo.

- Limitação: No momento em que a FSM interage com o mundo real (Sensores, BD, APIs), você entra no domínio da IO Monad. A formalização não pode garantir que o banco de dados estará disponível ou que o sensor não enviará um dado corrompido.
- O Risco: A segurança de tipos termina na "fronteira" entre o código puro e o código de I/O.

Eliminar Erros de Requisito (Semântica vs. Sintaxe)

O compilador é um verificador de regras, não um leitor de mentes.

- Limitação: Se você formalizar que a máquina deve ir para o estado Erro quando deveria ir para Sucesso, o compilador aceitará isso como uma "verdade matemática".
- Conclusão: A formalização garante que o código obedece ao modelo, mas não garante que o modelo obedece à realidade do negócio se o levantamento de requisitos estiver errado.

Rigor Categórico Total ("Hask is not a Category")

Conforme mencionado no documento de Duarte Maia, existe um debate teórico sobre o Haskell não ser uma categoria perfeita (devido a valores como undefined ou loops infinitos, chamados de bottom ou ⊥)

- Limitação: Em casos extremos, as leis monádicas que sustentam a sua composição (Kleisli) podem ser violadas por funções que não terminam ou que lançam exceções não capturadas.

O LIMITE DA FORMALIZAÇÃO:

O que a sua Formalização Resolve

Estados Inválidos: Proíbe estados que não existem no ADT.

Transições Proibidas: O compilador recusa o código se a transição não for permitida.

Exaustividade: Garante que todos os eventos conhecidos são tratados.

Purity: Isola o estado de mudanças acidentais.

O que permanece como Desafio/Impossível

Lógica Errada: Não impede que você defina uma regra de negócio burra.

Falhas Externas: Não controla se a rede cai durante a execução da transição.

Eventos Inesperados: Não lida com "desconhecidos desconhecidos" fora do ADT.

Performance: Não garante que a FSM será rápida ou eficiente em memória.

Dica para a defesa: Se alguém perguntar sobre isso, responda que "a formalização elimina toda uma classe de erros técnicos e estruturais, permitindo que os testes e a atenção do desenvolvedor se foquem exclusivamente na lógica de negócio e na integração com o mundo real, que são os problemas que a matemática pura não pode resolver sozinha."

PROGRESSO PARCIAL

5. Visão Geral

- Implementação estruturada de FSMs em Haskell, do modelo ingênuo até a verificação formal
- Correspondência explícita entre código-fonte e arcabouço teórico (Hutton, Milewski, Perone)
- Culminou na aplicação do framework CREM para verificação formal em tempo de compilação

5.1 Tipos Algébricos ↔ Conjuntos Q e Σ

- data State = Locked | Unlocked e data Input = Coin | Push
- Fundamentação: Milewski — tipo Bool = coproduto de dois objetos terminais (2 = 1+1)
- State e Input são estruturalmente isomorfos a Bool na categoria Hask
- Confirma que a finitude de Q e Σ é preservada formalmente, não só por convenção

```
data State = Locked | Unlocked deriving (Show, Eq)
data Input = Coin | Push deriving (Show, Eq)
```

5.2 A Função de Transição δ como Morfismo em Hask

- transition :: State → Input → State via pattern matching exaustivo
- Currying (Milewski): (a,b) → c ≡ a → b → c → correspondência direta com δ : Q × Σ → Q
- Completude Funcional garantida pela exaustividade do pattern matching (Hutton)
- Type safety: erros de tipo detectados antes da execução (compilação)

```
transition :: State → Input → State
transition Locked   Coin = Unlocked
transition Locked   Push = Locked
transition Unlocked Coin = Unlocked
transition Unlocked Push = Locked
```

5.3 Sequenciação Inicial e Validação Empírica

- run = foldl transition
- Execução sobre [Coin, Push, Push, Coin, Coin, Push] a partir de Locked → resultado determinístico Locked
- Base: pureza referencial (Milewski) — mesmo argumento, mesmo resultado, sem efeitos colaterais

```
run :: State → [Input] → State
run = foldl transition
```

5.4 Da Composição Ingênua à Necessidade de uma Abstração Monádica

- foldl só funciona se a transição não carregar informação adicional (ex.: mensagem de saída)
- Identificação teórica: transição pura vira função "embelezada" Q × Σ → M Q (Milewski)
- Formalização via Categoria Kleisli: morfismos A → M B, composição pelo operador fish (>=>)

```
(<=<) :: (a -> Writer b) -> (b -> Writer c) -> (a -> Writer c)
m1 >=> m2 = \x -> let (v, s1) = m1 x
                      (z, s2) = m2 y
                  in (z, s1 ++ s2)

return :: a -> Writer a
return x = (x, "")
```

5.5 Reformulação Teórica da FSM sobre a State Monad

- Baseado em Hutton: newtype ST a = S (State → (a, State))
- Instâncias de Functor, Applicative e Monad para ST
- transicaoM :: Input → ST Output — estado do autômato encapsulado e "threaded" de forma pura

```haskell
newtype ST a = S (State → (a, State))

app :: ST a → State → (a, State)
app (S st) x = st x

instance Functor ST where
    fmap g st = S (\s → let (x, s') = app st s in (g x, s'))

instance Applicative ST where
    pure x = S (\s → (x, s))
    stf <*> stx = S (\s → let (f, s')  = app stf s
                              (x, s'') = app stx s'
                          in (f x, s''))

instance Monad ST where
    st >>= f = S (\s → let (x, s') = app st s in app (f x) s')
```

A partir dessa estrutura, o projeto derivou a formalização de uma transição monádica para a FSM, em que o "estado" de Hutton (2016) é instanciado pelo estado do autômato (Q) e o valor produzido a cada passo é a saída associada à transição

```
transicaoM :: Input → ST Output
transicaoM evt = S (\q → let q' = transition q evt
                         in (mensagem q evt, q'))
```

5.6 Sequenciação via Notação do e a Função mapM

- Generalização de foldl para o contexto monádico: mapM :: Monad m ⇒ (a → m b) → [a] → m [b]
- mapM transicaoM eventos :: ST [Output] — produz lista de saídas + estado final
- Notação do (Oliveira) preserva pureza referencial com sintaxe imperativa

```
mapM :: Monad m ⇒ (a → m b) → [a] → m [b]
mapM f []     = return []
mapM f (x:xs) = do y  ← f x
                   ys ← mapM f xs
                   return (y:ys)
```

5.7 Fundamentação Categorial: o Monad como Monoide na Categoria dos Endofuntores

- Milewski: monad = endofunctor T + transformações naturais $\mu : T \circ T \to T$ e $\eta : I \to T$
- "Monad is just a monoid in the category of endofunctors" (Mac Lane)
- Garante associatividade e elemento neutro na composição de transições via $\gg=$

Em síntese, o resultado desta etapa é de natureza primariamente teórica e formal.

5.8 Estudo de Caso: Topologia Promovida ao Nível de Tipos

- Framework CREM (Perone) aplicado ao estudo de caso da catraca
- Topologia como valor de primeira classe, promovida a tipo via singletons
- Extensão do sistema de tipos (Hutton): regra de negócio da FSM vira propriedade do próprio tipo

```
$(singletons [d|
  data TurnstileVertex = Locked | Unlocked
    deriving (Show, Eq)
|])

turnstileTopology :: Topology TurnstileVertex
turnstileTopology = Topology
  [ (Locked, [Unlocked])
  , (Unlocked, [Locked])
]
```

5.9 Verificação Formal de Segurança de Tipos em Tempo de Compilação

- turnstileAction genérica sobre Applicative m, com prova implícita AllowedTransition
- Tentativa deliberada de transição inválida → rejeitada pelo GHC em tempo de compilação
- Erro de tipo decorre de violação de regra de negócio, não só de incompatibilidade estrutural

```
turnstileAction :: Applicative m
  ⇒ STurnstileVertex initialVertex
  → Input
  → ActionResult m TurnstileTopology STurnstileVertex initialVertex Output
turnstileAction SLocked Coin = pureResult "Unlocked (insert coin)" SUnlocked
turnstileAction SLocked Push = pureResult "Locked (push denied)" SLocked
turnstileAction SUnlocked Coin = pureResult "Unlocked (already open)" SUnlocked
turnstileAction SUnlocked Push = pureResult "Locked (passed through)" SLocked
```

5.10 Execução e Validação Cruzada

- cabal run sobre a mesma sequência de entradas → mesmo traço de execução
- Estado final: Locked — idêntico ao resultado da implementação ingênua (foldl)
- Validação cruzada entre duas formalizações estruturalmente distintas da mesma FSM

```
Coin → Unlocked (insert coin)
Push → Locked (passed through)
Push → Locked (push denied)
Coin → Unlocked (insert coin)
Coin → Unlocked (already open)
Push → Locked (passed through)
```

5.11 Limitações Identificadas e Encaminhamentos

- Topologia atual restringe apenas Q → Q (não o par completo Q × Σ → Q)
- Direção futura: uso de functores representáveis (Milewski) para indexar transições por Σ
- Pendente: geração automática de diagramas via módulo Render do CREM

5.12 Síntese Geral e Validação Preliminar das Hipóteses

- Completude Funcional → confirmada (pattern matching exaustivo)
- Segurança de Tipos → validada em dois níveis (Haskell base + regra de domínio via CREM)
- Composabilidade e Modulação → fundamentação teórica estabelecida; composição efetiva (Sequential/Parallel/Alternative/Feedback) é objetivo central da continuidade